FlashMLA & DeepSelect & DeepJIT 解读



https://gitcode.com/cann

CANN

目录

FlashMLA

DeepSelect

DeepJIT

稀疏 KV 的 Attention 计算

I_s = indices[s, :topk_length[s]] 中的有效索引

$$K_s = K[I_s] \qquad V_s = V[I_s]$$

$$P_s = a \cdot Q_s K_s^T$$

$$O_s = \text{softmax}(P_s) \cdot V_s$$

索引式省略单 KV head 维；softmax 沿所选 token 维归一化，K / V 共用 512 维

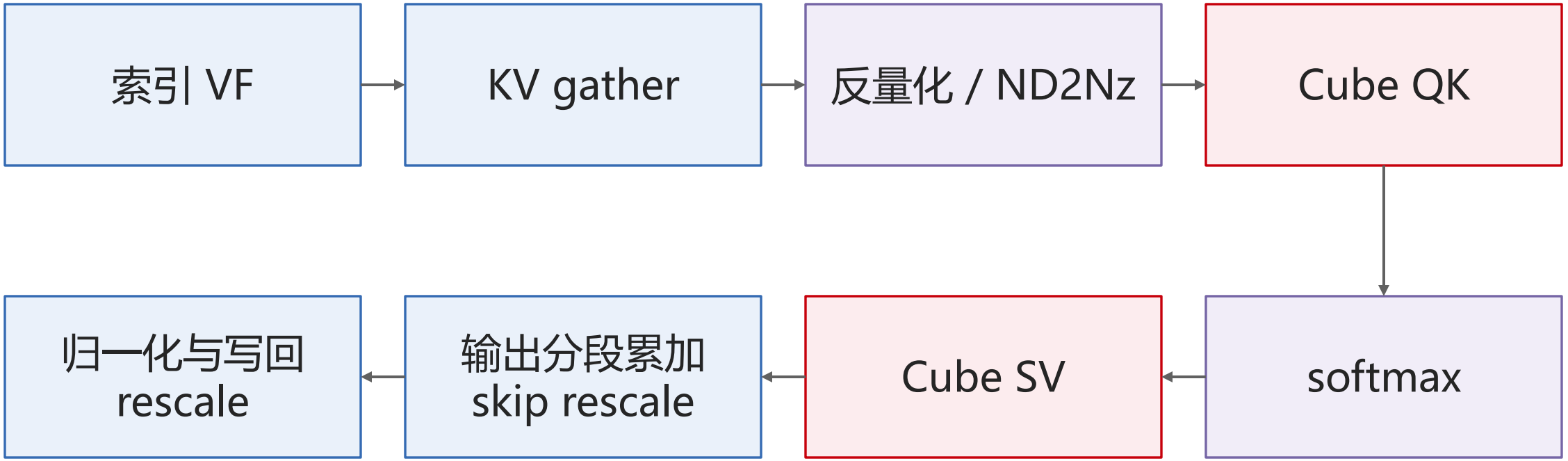
Prefill 与 Decode 接口

核心张量	flash_mla_sparse_fwd	flash_mla_with_kvcache
	Prefill：非量化	Decode：伪量化
Q / O，BF16	[S,64,512]	[B,1,64,512]
KV	[N,1,512] BF16	主 FP8；extra FP8 / FP4
indices，int32	[S,1,K] token 下标	[B,1,K] 物理 token 下标
有效 top-k	topk_length[S]	主 / extra 各自指定长度

64 query heads · 1 KV head · 512 维;

Decode s_q=1；逻辑多 query 由测试封装展平为 B×s_q；页大小 > 1，页内 token 连续

一次 Attention 的完整数据流



下排数据沿右向左流动；P 与输出分片经 FixPipe 进入 AIV 的 UB

基本块切分：每个AI核处理 64 个 head，64 个 token

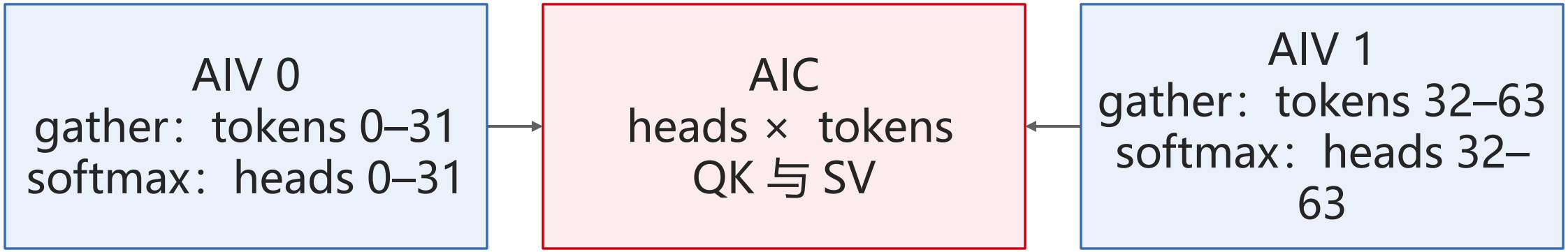
同一份 KV 供多个 head 使用；算存比随 head 变化

Heads per AIC	设计取舍	L0C 中的 O
32	算存比偏低，易受 KV 访存限制	64 KiB
64	M=64 已能充分利用 Cube	128 KiB
128	Buffer受限：O 占满 L0C，当前布局无法再放 P	256 KiB

O 占用 = H × 512 × 4 B；H=64 时 O + P = 144 KiB

1C2V 的任务划分

一个 query：64 heads × 512 head_dim，直接搬入cube核



每个 KV block 为 64 tokens；不同阶段沿不同矩阵轴切分

稀疏访存带来的问题

选中的 KV token 分散，先遇到访存瓶颈

topK indices [2, 9, 27, 41, ...] → GM / L2 中的分散记录

开销来源	为什么拖慢访存	对应优化
请求细碎	逐 token 搬运，指令发射次数多	① 两个 token 合成一条请求
单核并发受限	gather2 下，16 个请求覆盖 32 tokens	② 两个 AIV 各承担半块
data / scale 分离	量化缓存中，每个 token 需要两次读取	③ data 与 scale 相邻存放

CANN

稀疏访存优化①：gather2 合并请求

每个 AIV：32 tokens / 块，32 次单条请求变为 16 次双条请求

索引 VF
[9,2] 整理为 [2,9]

→

一个 MTE2 请求
n_burst = 2
base + stride

→

UB 连续两行
token 2 / token 9

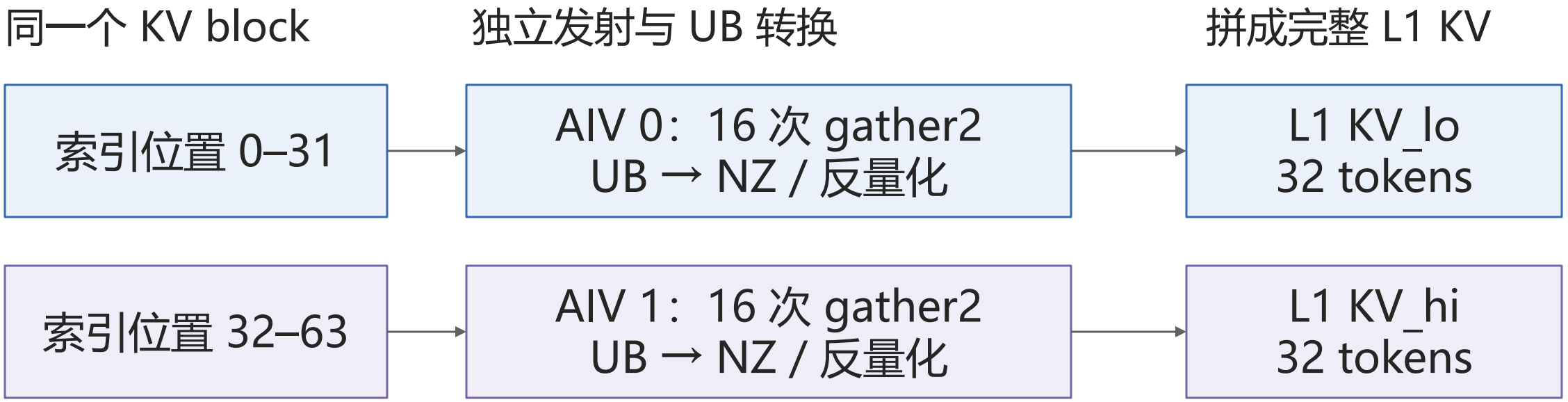
有效 token 数	搬运方式	Mask 处理
2	两条记录，一次请求	保留两项
1	复制有效项，stride=0	屏蔽另一项
0	n_burst=0，跳过搬运	两项均屏蔽

减少请求数与 Scalar 发射开销；数据量不减半。地址计算与 mask 由 VF 批量生成

CANN

稀疏访存优化②：两个 Vector 核分担一块 KV

64 个选中 token，沿 token 轴各分 32 个



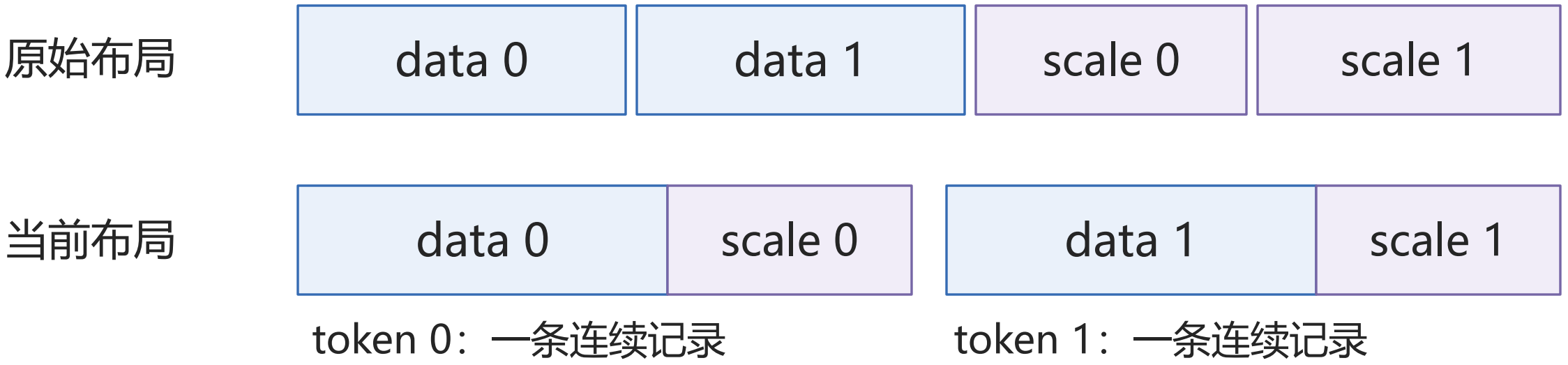
可并发覆盖：2 个 AIV × 16 个请求 × 2 tokens = 64 tokens

Cube 等两侧 L1_KV_BUF_FULL 后消费。两核各发请求，实际收益受共享带宽和延迟约束。

CANN

稀疏访存优化③：KV 记录布局

每个 token 的 data 与 scale 相邻，一次搬入完整记录



FP8：512 + 16 = 528 B　　FP4 extra：256 + 32 = 288 B

随后 VF 直接反量化并写入 BF16 NZ

Skip-scale①：数学原理

$$U(r) = \Sigma_j \exp[a(p_j - r)] \cdot V_j$$

$$l(r) = \Sigma_j \exp[a(p_j - r)] \qquad O = U(r) / l(r)$$

分子、分母共同的 exp(−ar) 抵消，r 不必每块更新

换基准：$\alpha = \exp[a(r_old - r_new)]$

$U \leftarrow \alpha U$，$l \leftarrow \alpha l$；保留 r 时 $\alpha = 1$

跳过历史 O 的换尺度；当前块仍做 QK、softmax 与 SV。实数代数等价，实现含浮点舍入

CANN

Skip-scale②：算法实现

每个 AIV：$\Delta = \max_h(m_new - r_old)$，阈值为 6/a

m_new = max(m_old, rowmax(p))；m 为真实最大值，r 为指数基准

决策	Δ ≤ 6/a：保留基准	Δ > 6/a：换基准
r_new	r_old	m_new
α	1	exp[a(r_old − m_new)]

S = exp[a(p − r_new)]　　l_new = αl_old + rowsum(S)

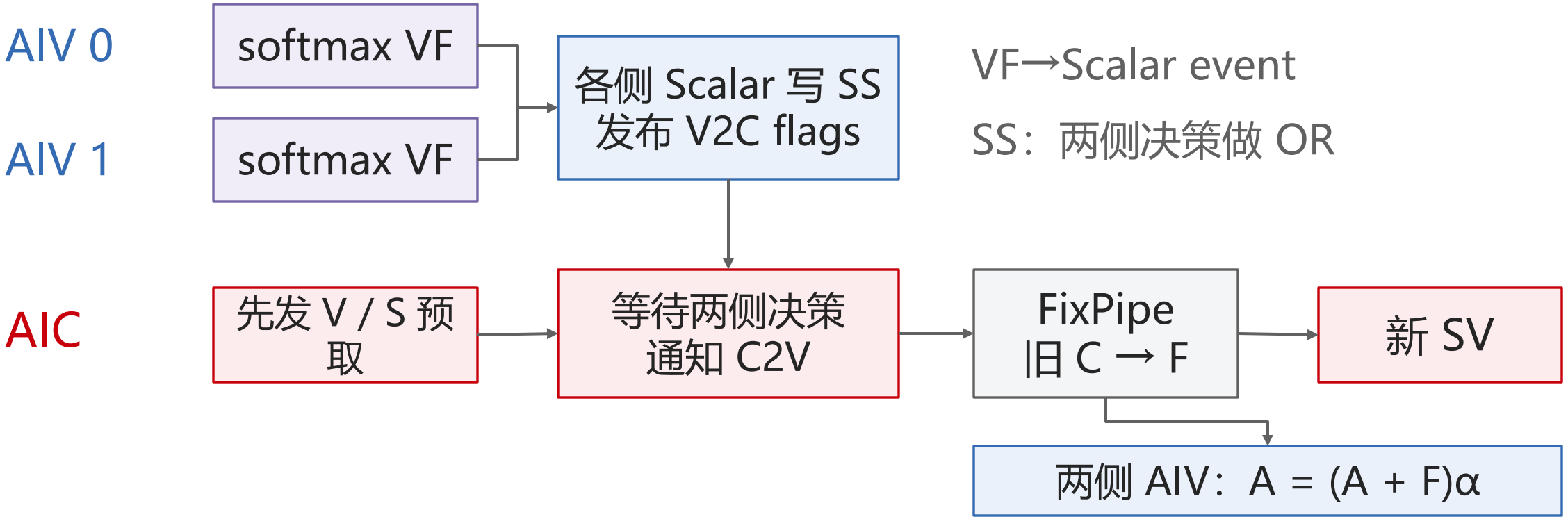
两侧决策做 OR：未触发则 C += S@V；触发先把旧 C 搬为 F

触发后 A = (A + F)α，C = S@V；最终 O = (A + C) / l

A：UB 历史段，C：L0C 当前段，F：搬出的旧段；首个历史段省去旧 A，空关注集合另行处理

Skip-scale③：决策与输出流水

展示触发 rescale 的分支，横向距离不代表耗时



四片全部搬出后才重写 L0C；末尾 VF 可与新 SV 重叠。未触发时跳过旧 O 搬出与缩放

Vector 核流水：一个 tick 内的七个动作

按列看七个动作，沿红色 B10 看同一 KV 块跨 tick 推进

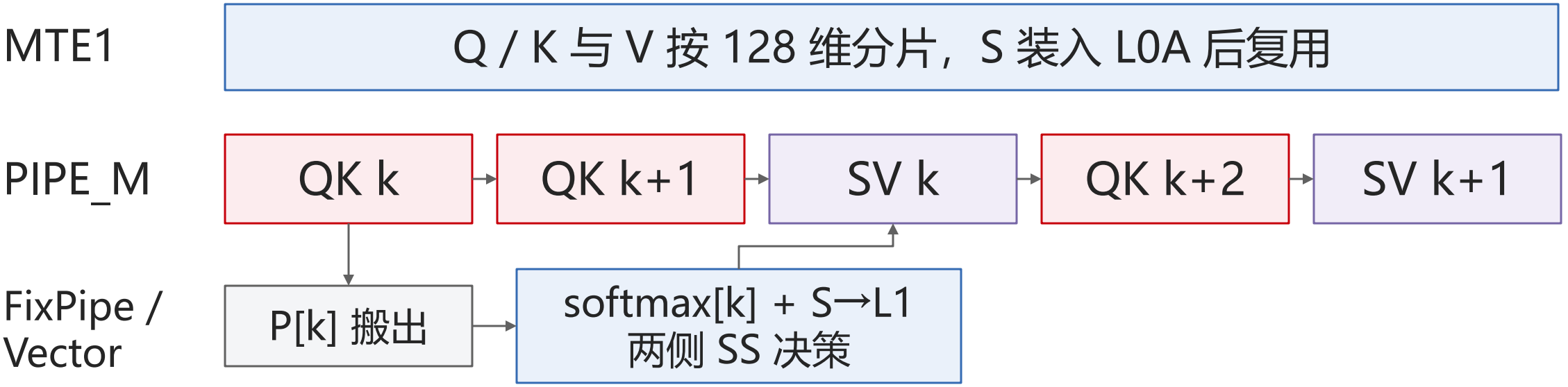
处理阶段 / 本 tick 的块号	i=5	i=6	i=7	i=8	i=9	i=10
① index 读取（i+5）	**B10**	B11	B12	B13	B14	B15
② ND2NZ + KV 的 UB → L1（i+3）	B8	B9	**B10**	B11	B12	B13
③ 等 P，softmax + S → L1（i+1）	B6	B7	B8	B9	**B10**	B11
④ index 预处理（i+5）	**B10**	B11	B12	B13	B14	B15
⑤ O scale，未 skip 时执行（i）	B5	B6	B7	B8	B9	**B10**
⑥ 等 VF，读取决策 → SS（i+1）	B6	B7	B8	B9	**B10**	B11
⑦ KV gather，GM → UB（i+5）	**B10**	B11	B12	B13	B14	B15

源码调度示意：①④仅组起点实际执行，⑤按需，格内为调度对象。tick 非等长耗时。
按 prefill 的⑥先于⑦分支排列，其他分支可交换⑥⑦。②③含 MTE3，⑦为 MTE2。

CANN

Cube 流水：用下一块 QK 留出 S 的准备窗口

QK(k+1) 先于 SV(k)，让 Vector 有时间完成 softmax 与 S 回传



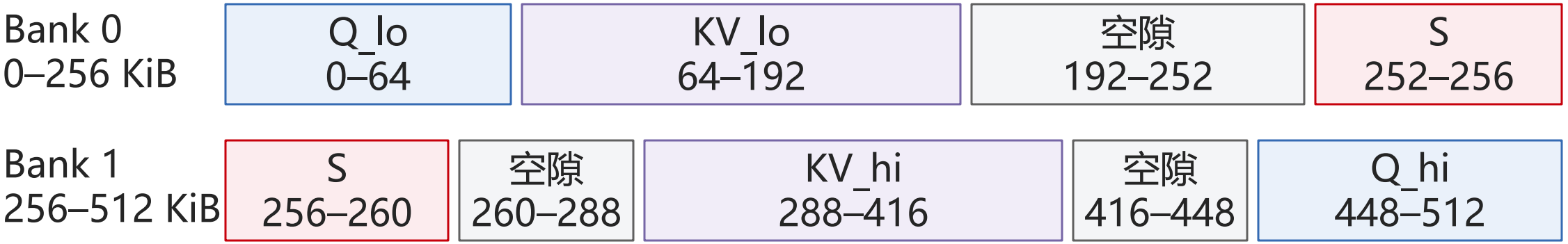
普通块：SV 直接累加 L0C O；需要 rescale 时，先等待旧 O 全部搬出

跨 query：预取 Q_next，先发 QK_next(0)，再做 SV_current(last)

CANN

Buffer 分配：带宽配平、存活期与安全复用

L1：把每份 Q、KV、S 分到两个 bank



配平带宽 Q / KV 各放一半，S 横跨 256 KiB 边界，避免只压一个 bank

按存活期定槽位 Q 双槽预取；KV 四槽保留至 SV；S 单份及时消费

消费完才归还 最后一次 Q / V / S → L0 完成后，事件允许生产者覆盖

地址单位 KiB，图宽不按比例。有效区 392 KiB，地址范围 512 KiB；空隙无需另建数据副本。

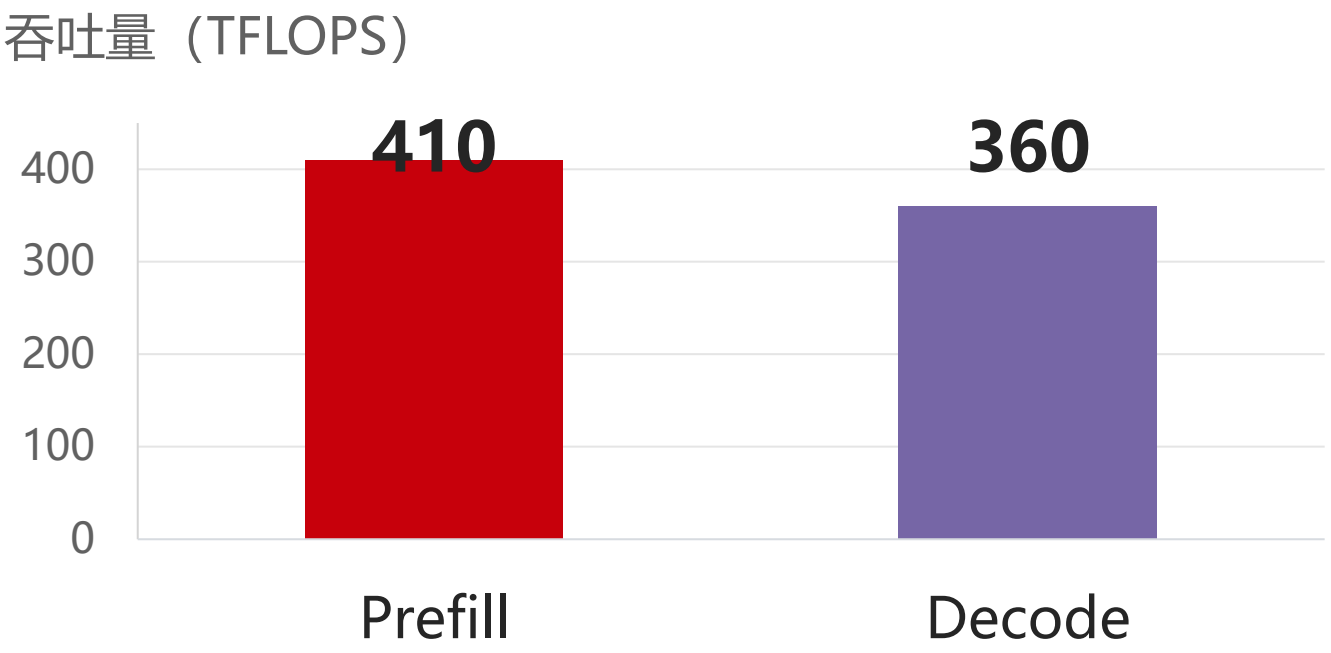
https://gitcode.com/cann

CANN

更加详细的技术报告和代码就在DeepSeek的FlashMLA仓库，感兴趣的同学可以直接去看。

典型业务场景性能

DeepSeek v4.1 典型业务场景



更加详细的技术报告和代码就在DeepSeek的FlashMLA仓库，感兴趣的同学可以直接去看。

https://gitcode.com/cann

CANN

目录

FlashMLA

DeepSelect

DeepJIT

算子的作用：逐行选出最大的 K 个值

对一行 x 的 L 个有效元素，设 L > K，且没有 NaN

$S \subseteq \{0, \ldots, L-1\}, \quad |S| = K$

$\min \{ x[i] : i \in S \} \geq \max \{ x[j] : j \notin S \}$

输出所选位置 indices，以及 values[r] = x[indices[r]]

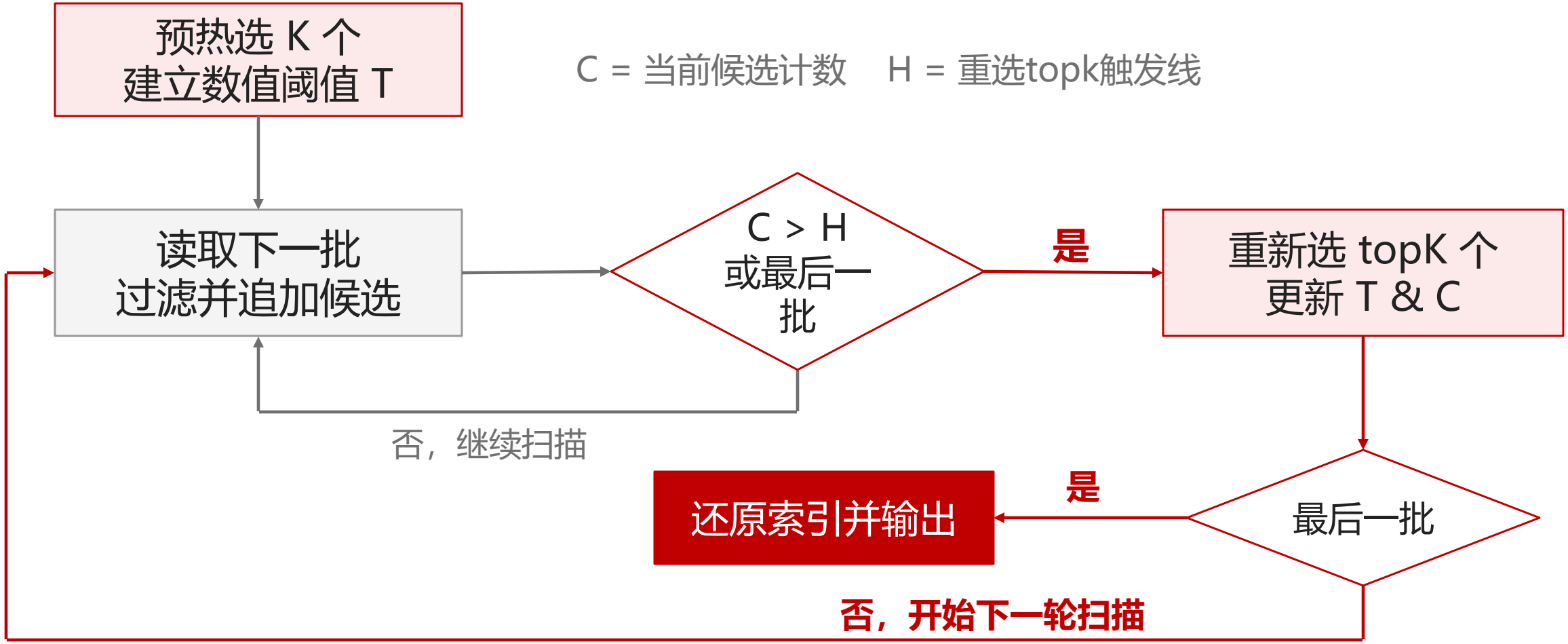
原始索引	0	1	2	3	4
输入值	3	**7**	1	**9**	5

K = 2：S = {1, 3}，values = [7, 9]

本例开启 sorted_index，按原始位置升序输出

CANN

算法流程：扫描与压缩构成多轮循环



CANN

TopK - RadixSelect：先看高 8 位，再看低 8 位

以 7 为例：BF16 二进制 0x40E0，正数翻转符号位后得到 排序key = 0xC0E0

高字节：位 15～8，0xC0 **低字节：位 7～0，0xE0**

16 位排序键	1	1	0	0	0	0	0	0	1	1	1	0	0	0	0	0

候选值及拆分后的字节（字节用十六进制表示），K = 4

数值	9	8	7	6	5	4	3	2
高 8 位	**C1**	**C1**	C0	C0	C0	C0	C0	C0
低 8 位	10	00	**E0**	**C0**	A0	80	40	00

① 高字节：C1 桶有 2 个值，第 4 大落在 C0 桶，还需选 2 个

② C0 桶内看低字节：E0 > C0 > A0 > 80 > 40 > 00，选中 7、6

入选 {9, 8, 7, 6}，第 4 大为 6，因此阈值 T = 6

CANN

一份数据的完整页映射

逻辑页	实际物理范围	虚拟页	页表值
0	P2 [4096, 6144)	0	32
1	P4 [8064, 10000)	1	63
2	P0 [0, 2048)	8	0
3	P1 [2048, 4096)	9	16
4	P3 [6144, 8064)	10	48

页表[虚拟页号] = 物理范围起点 ÷ 128

CANN

三种页：分别描述位置、顺序和编号

名称	它回答的问题	含义
逻辑页	这是第几次读页?	从 0 开始的读取顺序
物理页	数据原来在哪里?	输入行中的连续范围
虚拟页	内部怎样记录位置?	候选使用的页编号

以下统一使用：L = 10000，预热 2 页，演示 seed = 1

CANN

逻辑页映射：先安排读取顺序，再依次编号

示例：10000 个元素，每页最多 2048 个，预热 2 页，seed = 1

① 算页数：M = [10000 / 2048] = 5，共读取 5 页

② 安排尾部：尾片 P4 放在预热末尾，前段 P3 放在最后

③ 其余完整页按 seed 排列：本例得到 P2、P0、P1

逻辑页号	0	1	2	3	4
实际读取	P2	P4（尾片）	P0	P1	P3
阶段	预热	预热	扫描	扫描	扫描

逻辑页 0 = 第 1 次读取，实际读 P2 的 [4096, 6144)

逻辑页只编号读取顺序，输入数据不搬动；页数 ≤ 预热页数时顺序读

CANN

虚拟页：每批预留 8 页的编号区间

阶段	虚拟页编号	本例实际使用
预热	从 0 开始	0、1
预热后的空号	2...7	不使用
第一批扫描	8...15	8、9、10
第二批扫描	16...23	本例无需第二批

每批最多 8 × 2048 = 16384 个元素

整齐的编号区间，让同一批索引可以共用高 16 位

CANN

虚拟索引还原：跟踪原始位置 2300

原始位置 2300 位于 P1，页内偏移为 252

P1 对应逻辑页 3，内部编号是虚拟页 9

虚拟索引 = 9 × 2048 + 252 = 18684

输出时拆出：虚拟页 9，页内偏移 252

页表[9] = 16，原始位置 = 16 × 128 + 252 = 2300

目录

FlashMLA

DeepSelect

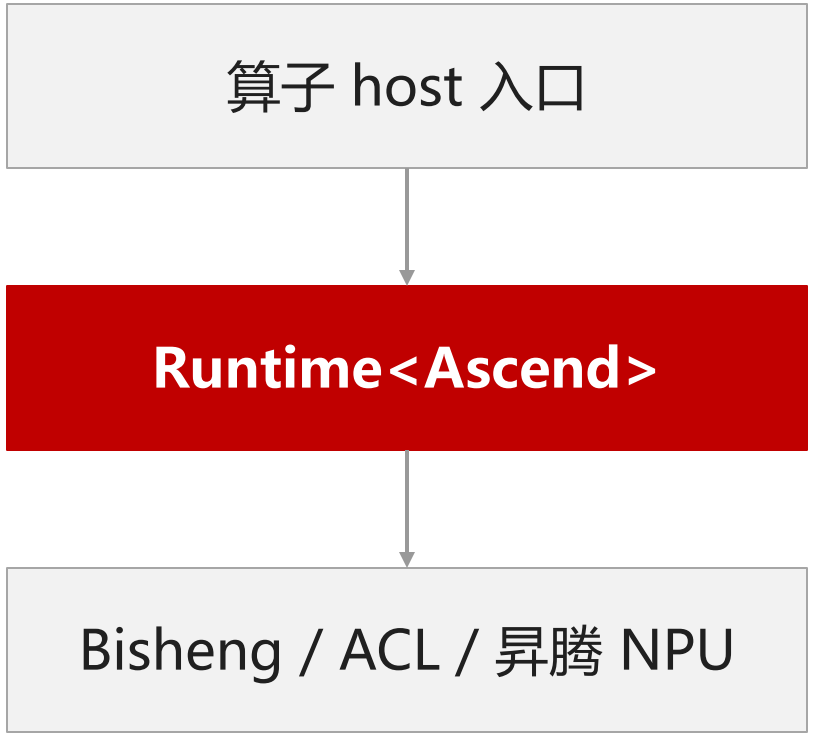
DeepJIT

DeepJIT 在算子库中的位置

轻量级JIT运行时
C++ 头文件库方式集成

算子库提供Kernel源码与编译运行参数
DeepJIT 管理编译、缓存和启动
包含内存、磁盘两级缓存

运行环境
Linux + CANN + torch_npu



使用 DeepJIT 示例

```
using namespace deep_jit;

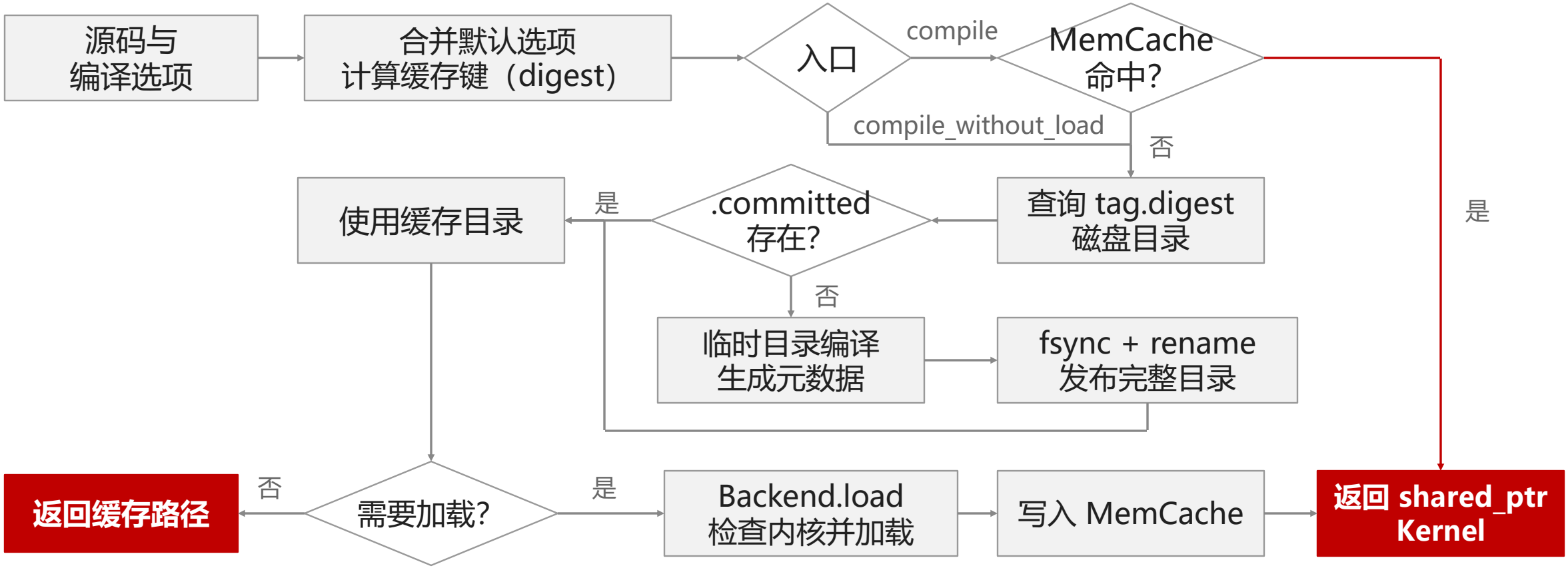
auto jit = create_lazy_jit<Ascend>(
    Config(root, "MYLIB", signature,
           {$root/"include"}, {"kernels/"}));

auto kernel = jit->compile(
    "increment", source);
jit->launch(kernel,
    {.num_blocks = 1}, output, 41);
```

root：算子路径
source：内核源码
output：算子参数

首次访问才构造 Runtime
先配置环境，再初始化

编译、加载与启动的完整流程



CANN

什么变化会产生新的缓存项

摘要输入	算子实现中的内容
源码与受追踪头文件	根源码 + include 内容摘要
有效编译选项	架构、优化参数、Bisheng 与链接参数
工具链版本	Bisheng 与 ld.lld 的版本输出
外部依赖签名	调用方提供的 extra_signature

仅递归追踪匹配前缀的尖括号 include

头文件摘要随 Runtime 缓存，修改头文件后需重建 Runtime

磁盘缓存的完整目录

```
<缓存根目录>/
├── cache/
│   └── <tag>.<digest>/
│       ├── kernel.asc
│       ├── kernel.o
│       ├── meta.json
│       ├── .committed
│       └── asm/（可选）
└── tmp/
    └── <uuid>/
```

kernel.asc

传入的昇腾内核源码

kernel.o

ACL 加载的内核二进制

meta.json

编译命令、配置及工具链版本

.committed

已提交标记，用于判定缓存命中

tmp/<uuid> 是构建用临时目录，完成后发布到 cache/<tag>.<digest>/

asm/ 仅在开启 dump_asm 时生成

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN